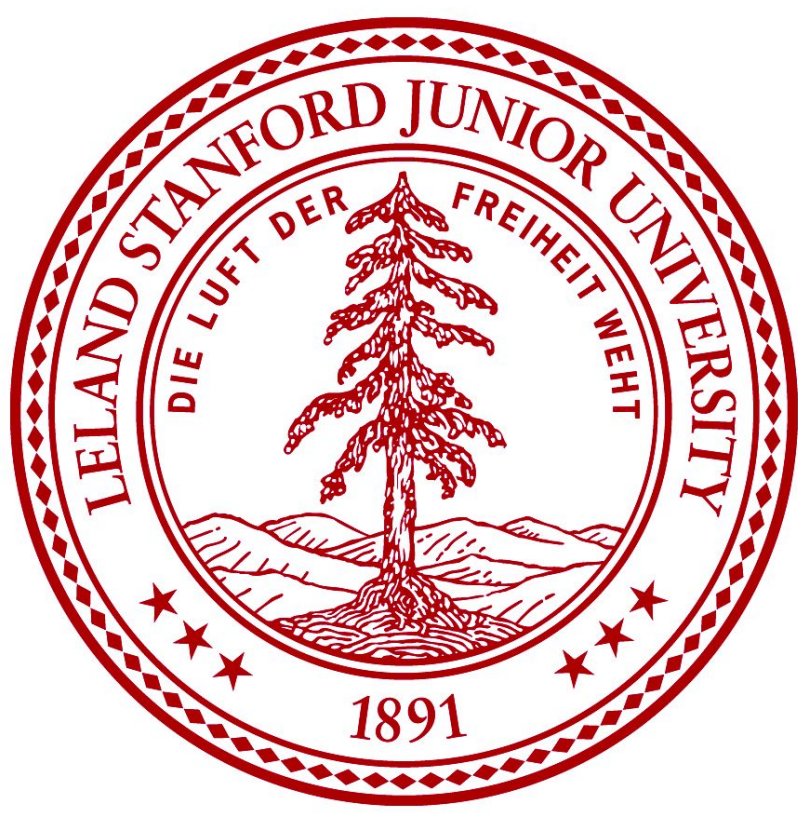
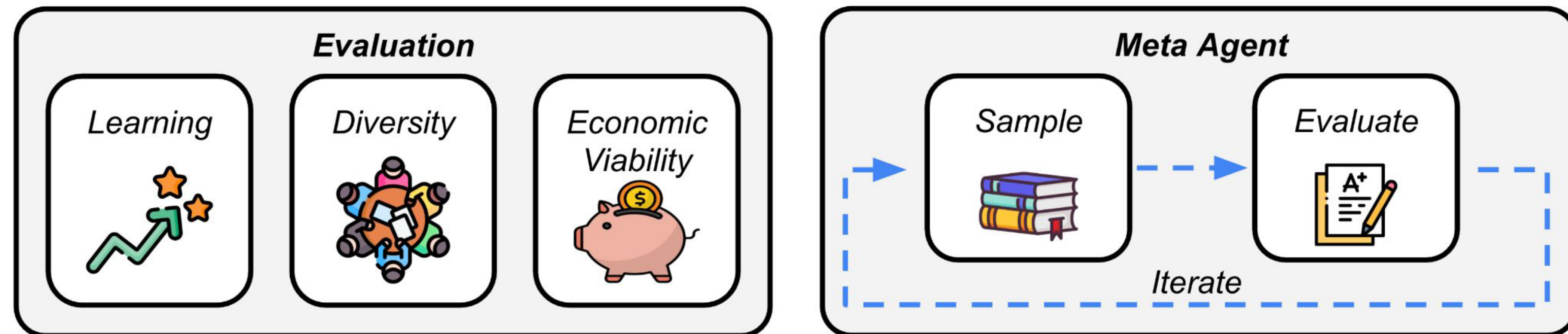




Inefficiencies of Meta Agents for Agent Design

Batu El, Mert Yuksekgonul, James Zou
Stanford University

Overview



Recent works began to automate the design of agentic systems using meta-agents that propose and iteratively refine new agent architectures. In this paper, we examine three key challenges in a common class of meta-agents.

- 1 Meta Learning.** We investigate how a meta-agent learns across iterations.
- 2 Diversity and Complementarity.** We explore diversity of the discovered agent architectures to assess potential for routing and collaboration.
- 3 Economic Viability.** We assess when automated design is economically viable.

Background

Algorithm 1 Meta Agent: *Sample-Evaluate-Iterate*

```

1:  $D_{\text{train}}$  # set of training examples
2:  $F$  # initial agents library
3:  $A = \{(f_{0_i}, s_{0_i}) \mid f_{0_i} \in F\}$  # archive
4: for  $t$  in  $[T]$  do
5:    $\hat{A} = \phi(A)$  # select current context
6:    $f_t \sim \Pi(\cdot \mid \hat{A})$  # sample, revise, debug
7:    $s_t = \text{eval}(f_t)$  # evaluate
8:    $A.\text{append}(f_t, s_t)$  # add to archive
9:   # iterate
10: end for

```

Our primary reference is ADAS (Hu et al., 2024), which has introduced meta-agent search with the idea of searching for agents in the code space. MAS-GPT (Ye et al., 2025) and ScoreFlow (Wang et al., 2025) develop meta-agents by training a model to dynamically generate multi-agent systems for a given query.

AgentSquare (Shang et al., 2025) and Archon (Saad-Falcon et al., 2024) explore modular agent architectures and use discrete module recombination to efficiently search design spaces. AutoFlow (Li et al., 2024), Weak-for-Strong (Nie et al., 2025), and ADAS (Hu et al., 2024) use a meta agent that follows the sample-evaluate-iterate paradigm (Algorithm 1).

Meta Learning

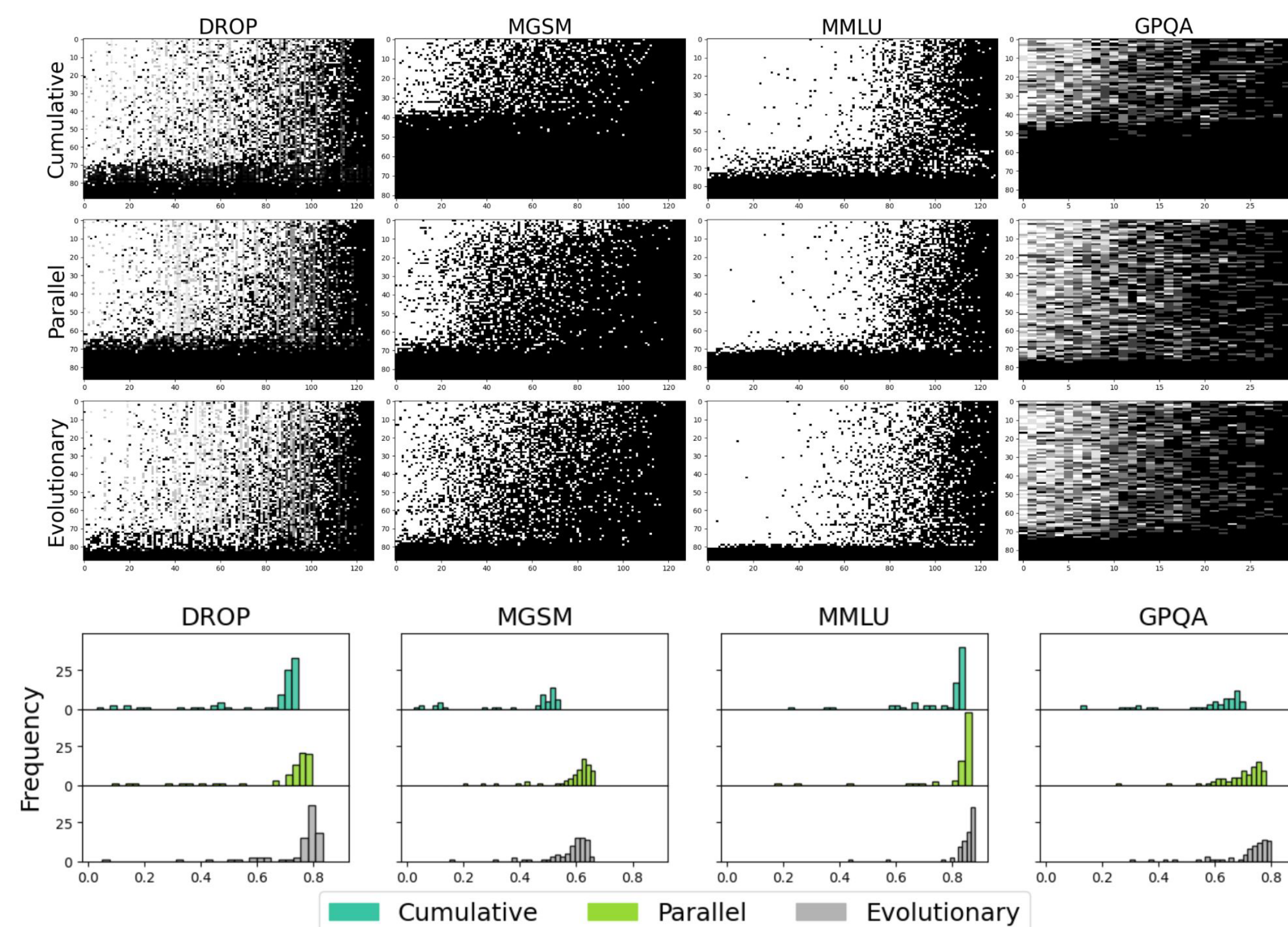
Dataset	Best Agent			Best-5 Avg.			Best-10 Avg.			Best-15 Avg.			Test Performance (Best Agent)			
	C	P	E	C	P	E	C	P	E	C	P	E	I	C	P	E
DROP	71.4	72.5	74.4	68.1	69.3	71.5	66.6	66.8	69.7	64.9	64.9	68.2	64.8	71.9	72.6	73.2
	(2.0)	(4.2)	(3.2)	(1.1)	(1.2)	(4.5)	(0.8)	(1.1)	(4.7)	(0.6)	(1.8)	(4.7)	(1.3)	(3.2)	(7.8)	(5.1)
MGSM	41.4	56.2	56.5	32.5	48.4	50.4	27.4	43.4	46.0	22.4	39.8	42.7	38.4	41.2	51.8	53.5
	(6.2)	(10.5)	(4.7)	(13.8)	(9.8)	(0.8)	(16.6)	(7.9)	(1.6)	(17.1)	(5.3)	(2.5)	(2.8)	(4.8)	(7.6)	(2.0)
MMLU	74.7	76.3	76.6	73.0	73.8	74.8	70.3	72.4	73.7	68.0	71.1	72.7	62.8	66.2	67.8	65.8
	(2.0)	(1.6)	(2.7)	(2.1)	(2.4)	(2.7)	(5.2)	(2.5)	(2.4)	(8.0)	(3.0)	(2.3)	(2.3)	(4.2)	(0.8)	(3.3)
GPQA	32.3	35.2	33.8	26.4	32.2	31.2	22.5	30.4	29.8	20.7	29.1	28.8	30.0	29.7	31.3	28.5
	(2.6)	(2.8)	(2.2)	(8.7)	(1.5)	(0.9)	(12.4)	(1.1)	(0.8)	(13.2)	(0.6)	(0.5)	(2.4)	(2.7)	(0.0)	(3.1)
Avg.	55.0	60.0	60.3	50.0	55.9	57.0	46.7	53.2	54.8	44.0	51.2	53.1	49.0	52.2	55.9	55.2

We experiment with three instantiations of context curation.

- 1 Cumulative (C).** The generation of the next agent is conditioned on all the previously discovered architectures.
- 2 Parallel (P).** The generation of the next agent is conditioned only on the initial library. Hence, the meta-agent ignores the previously designed architectures, effectively parallel sampling the new agents.
- 3 Evolutionary (E).** The generation of the next agent is conditioned on a subset of 7 agents selected based on the evaluation scores (top-k selection strategy) from the archive.

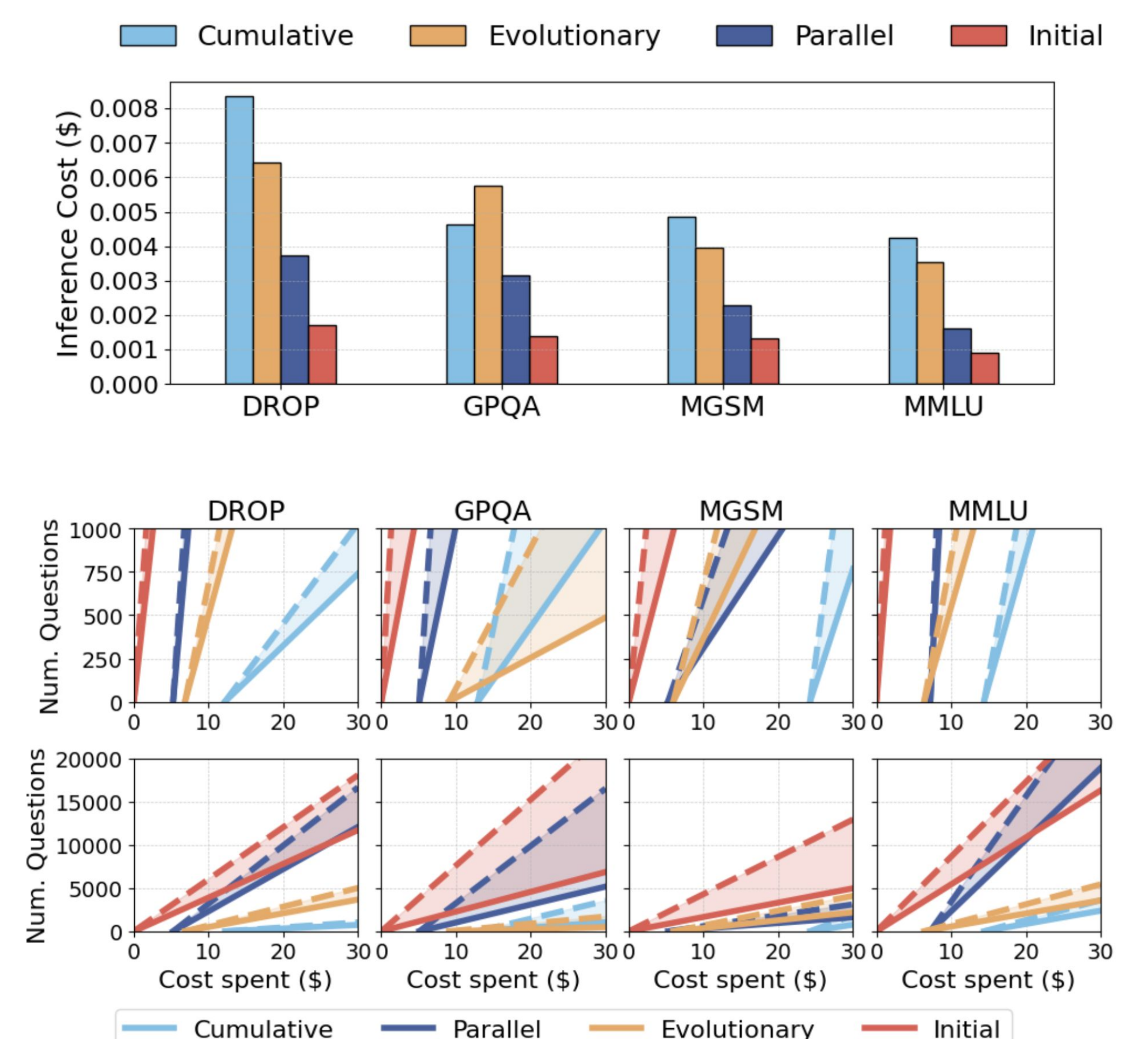
Finding. We find that simply expanding the context with all previous agents, as proposed by previous works, performs worse than ignoring prior designs entirely. We show that the performance improves with an evolutionary approach.

Diversity and Complementarity



Finding. Although the meta-agent designs multiple agents during training, it typically commits to a single agent at test time. We find that the designed agents have low behavioral diversity, limiting the potential for their complementary use.

Economic Viability



Finding. We find that only in a few cases the overall cost of designing and deploying the agents is lower than that of human-designed agents when deployed on over 15,000 examples. In contrast, the performance gains for other datasets do not justify the design cost, regardless of scale.